Assignment 1: Apply LDA Algorithm on Iris Dataset and classify which species a given flower belongs to.

In [ ]:
```
%pip install pandas numpy matplotlib scikit-learn openpyxl requests seaborn
"gymnasium[toy-text]" statsmodels
```

In [ ]:
```
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset from CSV
df = pd.read_csv("iris.csv")

# Separate features and target
X = df.drop("species", axis=1)
y = df["species"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Apply LDA
lda = LinearDiscriminantAnalysis()

# Train model
lda.fit(X_train, y_train)

# Predict species
y_pred = lda.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Predict a new flower
flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = lda.predict(flower)

print("Predicted species:", prediction[0])
```

```
Accuracy: 1.0
Predicted species: setosa
```

Assignment 2: Data Loading, Storage and File Format Handling Using Python

In [ ]:
```
import pandas as pd
```

```
import matplotlib.pyplot as plt

# 1. Load data from different formats
csv = pd.read_csv("sales.csv")
excel = pd.read_excel("sales.xlsx")
json = pd.read_json("sales.json")

# 2. Combine all datasets
df = pd.concat([csv, excel, json], ignore_index=True)

# 3. Explore data
print("First five rows:")
print(df.head())

print("\nDataset information:")
print(df.info())

print("\nMissing values:")
print(df.isnull().sum())

print("\nStatistical summary:")
print(df.describe(include="all"))

# 4. Clean data
df = df.drop_duplicates()

# Fill missing numeric values with column means
df = df.fillna(df.mean(numeric_only=True))

# Fill missing text values
df = df.fillna("Unknown")

# 5. Data transformation
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
df["Sales"] = df["Sales"].fillna(df["Sales"].mean())

# 6. Analyze sales
print("\nTotal Sales:", df["Sales"].sum())
print("\nAverage Sales by Category:")
print(df.groupby("Category")["Sales"].mean())

# 7. Visualize sales by category
df.groupby("Category")["Sales"].sum().plot(kind="bar")
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.show()
```

```
First five rows:
   OrderID   Category    Sales
0     5258  Furniture  1525.69
1     5605  Furniture   632.78
```

```
2     7795  Furniture  1021.86
3     9553  Groceries      NaN
4     1313  Furniture  1927.15

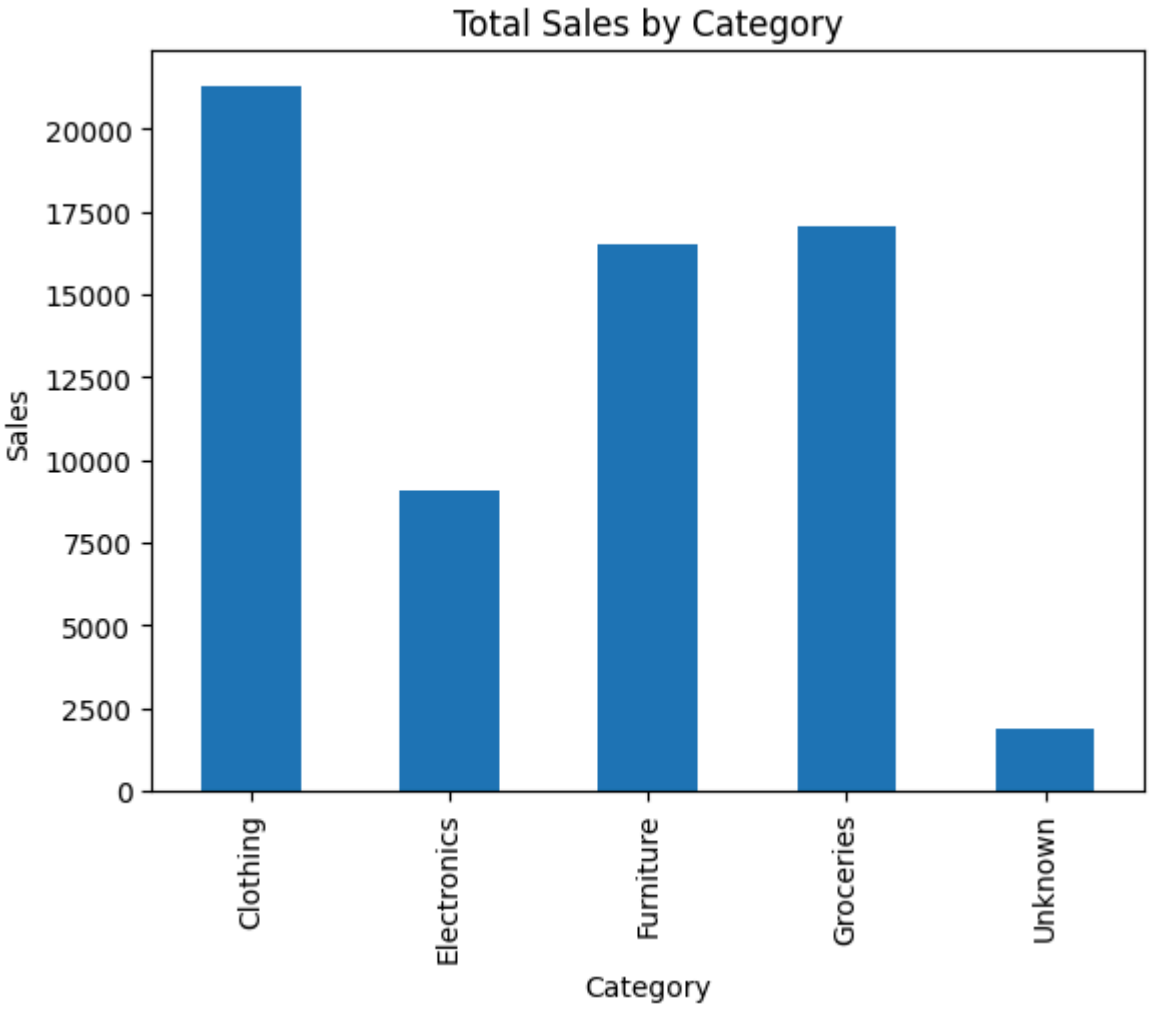
Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 61 entries, 0 to 60
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   OrderID   61 non-null     int64
 1   Category  60 non-null     str
 2   Sales     60 non-null     float64
dtypes: float64(1), int64(1), str(1)
memory usage: 1.6 KB
None

Missing values:
OrderID     0
Category    1
Sales       1
dtype: int64

Statistical summary:
            OrderID  Category        Sales
count     61.000000        60    60.000000
unique          NaN         4          NaN
top             NaN  Clothing          NaN
freq            NaN        20          NaN
mean    5087.770492       NaN  1090.846000
std     2673.853071       NaN   538.718763
min     1248.000000       NaN   102.830000
25%     2690.000000       NaN   666.847500
50%     4809.000000       NaN  1092.910000
75%     7610.000000       NaN  1513.427500
max     9934.000000       NaN  1949.570000

Total Sales: 65789.92881355932

Average Sales by Category:
Category
Clothing       1120.538421
Electronics    1005.690000
Furniture      1031.527500
Groceries      1139.202588
Unknown        1856.010000
Name: Sales, dtype: float64
```



Assignment 3: Regression Analysis

Predict the price of the Uber ride from a given pickup point to the agreed drop-off location. Perform following tasks:

1. Pre-process the dataset.
2. Identify outliers.
3. Check the correlation.
4. Implement linear regression and ridge, Lasso regression models.
5. Evaluate the models and compare their respective scores like R2, RMSE, etc.

In [ ]:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score, mean_squared_error

# 1. Load dataset
df = pd.read_csv("uber.csv")
```

```
# 2. Preprocessing
df = df.dropna()
df = df[df["fare_amount"] > 0]

# Select features and target
X = df[["pickup_longitude", "pickup_latitude", "dropoff_longitude",
"dropoff_latitude", "passenger_count"]]
y = df["fare_amount"]

# Remove invalid coordinate values
valid = (
    X["pickup_longitude"].between(-180, 180) &
    X["pickup_latitude"].between(-90, 90) &
    X["dropoff_longitude"].between(-180, 180) &
    X["dropoff_latitude"].between(-90, 90) &
    X["passenger_count"].between(1, 6)
)
X = X[valid]
y = y[valid]

# 3. Identify and remove fare outliers using IQR
Q1 = y.quantile(0.25)
Q3 = y.quantile(0.75)
IQR = Q3 - Q1

valid = (y ⩾ Q1 - 1.5 * IQR) & (y ⩽ Q3 + 1.5 * IQR)
X = X[valid]
y = y[valid]

print("Outliers removed using IQR")

# 4. Check correlation
data = X.copy()
data["fare_amount"] = y
print("\nCorrelation:")
print(data.corr())

# 5. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 6. Train three regression models
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": make_pipeline(StandardScaler(), Ridge(alpha=1.0)),
    "Lasso Regression": make_pipeline(StandardScaler(), Lasso(alpha=0.1))
}

# 7. Evaluate and compare models
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
```

```
    r2 = r2_score(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))

    print("\n", name)
    print("R2 Score:", round(r2, 3))
    print("RMSE:", round(rmse, 3))
```

```
Outliers removed using IQR

Correlation:
                   pickup_longitude  pickup_latitude  dropoff_longitude  \
pickup_longitude           1.000000        -0.980169       9.550712e-01
pickup_latitude           -0.980169         1.000000      -9.426733e-01
dropoff_longitude          0.955071        -0.942673       1.000000e+00
dropoff_latitude          -0.942984         0.962609      -9.807235e-01
passenger_count           -0.000739        -0.000801      -8.524098e-07
fare_amount                0.000134        -0.000590      -9.634702e-04

                   dropoff_latitude  passenger_count  fare_amount
pickup_longitude          -0.942984    -7.387182e-04     0.000134
pickup_latitude            0.962609    -8.007080e-04    -0.000590
dropoff_longitude         -0.980724    -8.524098e-07    -0.000963
dropoff_latitude           1.000000    -1.983893e-03     0.000544
passenger_count           -0.001984     1.000000e+00     0.012223
fare_amount                0.000544     1.222314e-02     1.000000

 Linear Regression
R2 Score: 0.0
RMSE: 4.125

 Ridge Regression
R2 Score: 0.0
RMSE: 4.125

 Lasso Regression
R2 Score: -0.0
RMSE: 4.125
```

Assignment 4: Analyzing Weather Data from OpenWeatherMap API

In [7]:

```
import requests
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Get weather data
API_KEY = "6fb5538ad63887d80044e8496a044b74"
city = "Pune"

url = "https://api.openweathermap.org/data/2.5/forecast"
```

```
params = {"q": city, "appid": API_KEY, "units": "metric"}

response = requests.get(url, params=params)
data = response.json()

if response.status_code == 200:
    # 2. Extract weather data using a for loop
    weather_list = []

    for x in data["list"]:
        record = {
            "Date": x["dt_txt"],
            "Temperature": x["main"]["temp"],
            "Humidity": x["main"]["humidity"],
            "Wind_Speed": x["wind"]["speed"],
            "Precipitation": x.get("rain", {}).get("3h", 0)
        }

        weather_list.append(record)

    # 3. Create DataFrame
    df = pd.DataFrame(weather_list)
    df["Date"] = pd.to_datetime(df["Date"])

    print(df.head())

    # 4. Statistics
    print("Average Temperature:", df["Temperature"].mean())
    print("Maximum Temperature:", df["Temperature"].max())
    print("Minimum Temperature:", df["Temperature"].min())

    # 5. Temperature graph
    plt.plot(df["Date"], df["Temperature"])
    plt.show()

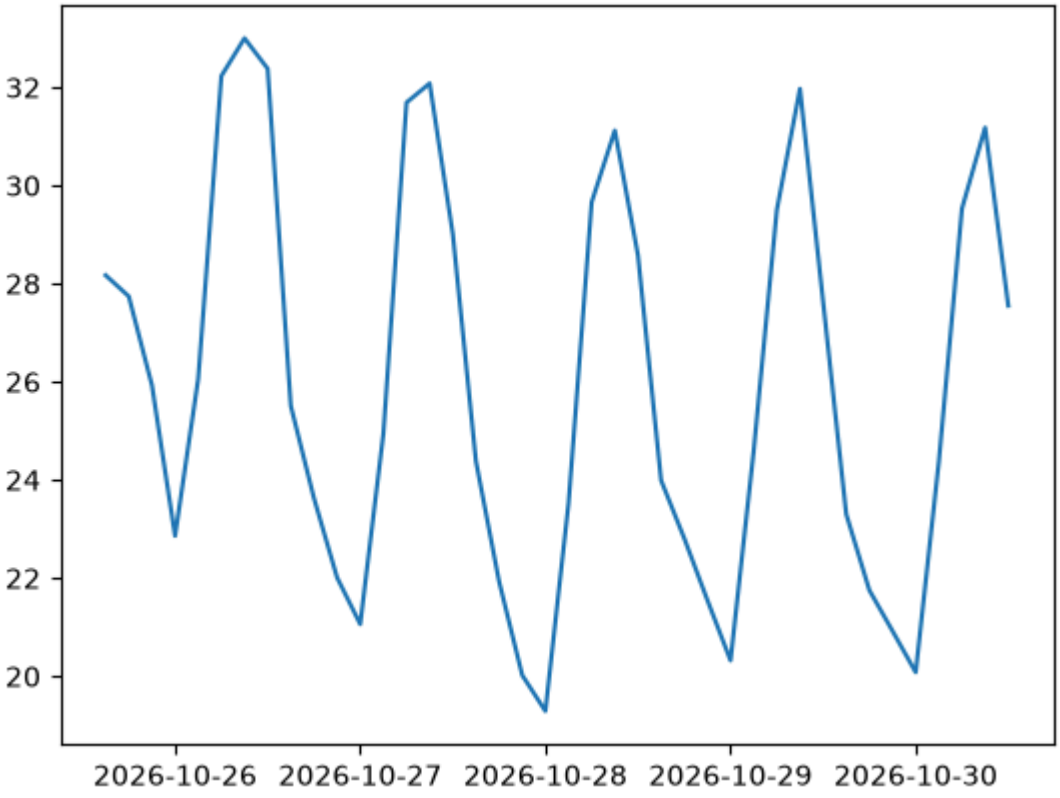
    # 6. Correlation heatmap
    sns.heatmap(df[["Temperature", "Humidity", "Wind_Speed",
"Precipitation"]].corr())
    plt.show()

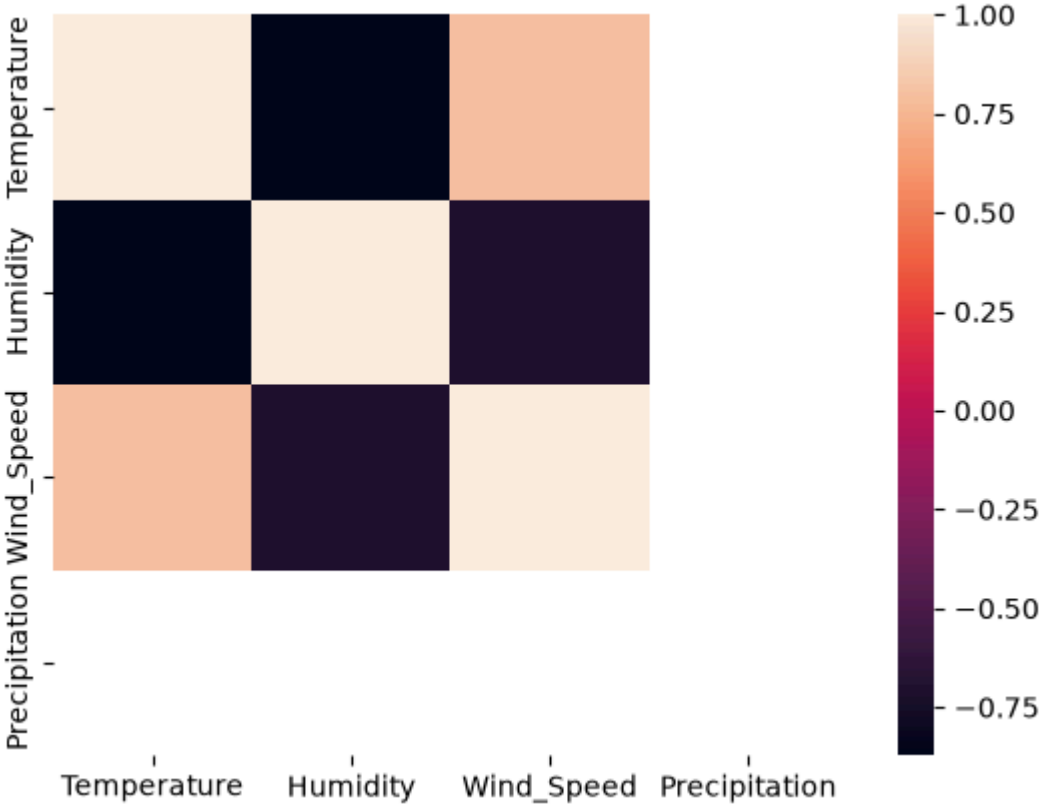
    # 7. Location details
    print("City:", data["city"]["name"])
    print("Latitude:", data["city"]["coord"]["lat"])
    print("Longitude:", data["city"]["coord"]["lon"])

else:
    print("Error:", data.get("message", "Unable to get weather data"))
```

	Date	Temperature	Humidity	Wind_Speed	Precipitation
0	2026-10-25 15:00:00	28.17	44	2.20	0
1	2026-10-25 18:00:00	27.74	47	1.79	0
2	2026-10-25 21:00:00	25.93	45	2.58	0

```
3 2026-10-26 00:00:00        22.86        39        1.51              0
4 2026-10-26 03:00:00        26.06        36        2.80              0
Average Temperature: 25.946000000000005
Maximum Temperature: 33.0
Minimum Temperature: 19.29
```






```
City: Pune
Latitude: 18.5196
```

```
Longitude: 73.8553
```

Assignment 5:

Implement K-Nearest Neighbours' algorithm on Social network ad dataset. Compute confusion matrix, accuracy, error rate, precision and recall on the given dataset.

In [ ]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score

# 1. Load dataset
df = pd.read_csv("Social_Network_Ads.csv")

# 2. Select features and target
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)

# 4. Feature scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# 5. Apply KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# 6. Predict
y_pred = knn.predict(X_test)

# 7. Evaluate model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

# 8. Display results
print("Confusion Matrix:\n", cm)
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

```
Confusion Matrix:
 [[33  7]
 [ 3 37]]
Accuracy: 0.875
Error Rate: 0.125
Precision: 0.8409090909090909
Recall: 0.925
```

Assignment 6:

Analyzing Customer Churn in a Telecommunications Company

In [ ]:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# 1. Load and explore data
df = pd.read_csv("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv")
print(df.head())
print(df.info())

# 2. Clean missing values and duplicates
df.drop_duplicates(inplace=True)
df = df.ffill()

# 3. Convert TotalCharges to numeric if present
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    df["TotalCharges"] =
df["TotalCharges"].fillna(df["TotalCharges"].median())

# 4. Handle outliers using IQR for numeric columns
for col in df.select_dtypes(include="number").columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    df[col] = df[col].clip(Q1 - 1.5 * IQR, Q3 + 1.5 * IQR)

# 5. Encode categorical columns
for col in df.select_dtypes(include="object").columns:
    df[col] = LabelEncoder().fit_transform(df[col].astype(str))

# 6. Split features and target
X = df.drop("Churn", axis=1)
y = df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 7. Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

# 8. Feature Engineering
df["AverageMonthlySpend"] = df["TotalCharges"] / (df["tenure"] + 1)
print(df.head())

# 9. Save cleaned dataset
df.to_csv("cleaned_telecom_churn.csv", index=False)
```

```
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService
\
0  2137-ZAMRH    Male              0      No         No       9          Yes
1  3665-MOOOQ  Female              0      No        Yes       9          Yes
2  4534-YYPTV  Female              0      No         No      58          Yes
3  1482-MXLKX    Male              0      No        Yes      36          Yes
4  8136-HNHHO  Female              0     Yes         No      43          Yes

  MultipleLines InternetService OnlineSecurity  ...     DeviceProtection  \
0            No              No             No  ...                   No
1           Yes     Fiber optic             No  ...                  Yes
2            No              No            Yes  ...  No internet service
3           Yes     Fiber optic             No  ...                   No
4            No     Fiber optic            Yes  ...  No internet service

           TechSupport          StreamingTV StreamingMovies        Contract
\
0  No internet service  No internet service             Yes  Month-to-month
1                   No  No internet service              No        Two year
2                   No                  Yes              No  Month-to-month
3  No internet service                  Yes             Yes        One year
4                   No  No internet service              No  Month-to-month

  PaperlessBilling              PaymentMethod MonthlyCharges  TotalCharges
\
0              Yes  Bank transfer (automatic)          43.19        395.23
1               No    Credit card (automatic)          66.19        588.17
2              Yes  Bank transfer (automatic)          22.83       1271.29
3              Yes           Electronic check          47.18        1694.8
4               No               Mailed check         116.84       5196.56

  Churn
0    No
1    No
2    No
3   Yes
4    No

[5 rows x 21 columns]
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        1000 non-null   str
```

```
 1   gender            1000 non-null   str
 2   SeniorCitizen     1000 non-null   int64
 3   Partner           1000 non-null   str
 4   Dependents        1000 non-null   str
 5   tenure            1000 non-null   int64
 6   PhoneService      1000 non-null   str
 7   MultipleLines     1000 non-null   str
 8   InternetService   1000 non-null   str
 9   OnlineSecurity    1000 non-null   str
 10  OnlineBackup      1000 non-null   str
 11  DeviceProtection  1000 non-null   str
 12  TechSupport       1000 non-null   str
 13  StreamingTV       1000 non-null   str
 14  StreamingMovies   1000 non-null   str
 15  Contract          1000 non-null   str
 16  PaperlessBilling  1000 non-null   str
 17  PaymentMethod     1000 non-null   str
 18  MonthlyCharges    1000 non-null   float64
 19  TotalCharges      1000 non-null   str
 20  Churn             1000 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 164.2 KB
None
   customerID  gender  SeniorCitizen  Partner  Dependents  tenure  \
0         114       1              0        0           0       9
1         280       0              0        0           1       9
2         393       0              0        0           0      58
3          52       1              0        0           1      36
4         798       0              0        1           0      43

   PhoneService  MultipleLines  InternetService  OnlineSecurity  ...  \
0             1              0                2               0  ...
1             1              2                1               0  ...
2             1              0                2               2  ...
3             1              2                1               0  ...
4             1              0                1               2  ...

   TechSupport  StreamingTV  StreamingMovies  Contract  PaperlessBilling  \
0            1            1                2         0                 1
1            0            1                0         2                 0
2            0            2                0         0                 1
3            1            2                2         1                 1
4            0            1                0         0                 0

   PaymentMethod  MonthlyCharges  TotalCharges  Churn  AverageMonthlySpend
0              0           43.19        395.23      0            39.523000
1              1           66.19        588.17      0            58.817000
2              0           22.83       1271.29      0            21.547288
3              2           47.18       1694.80      1            45.805405
4              3          116.84       5196.56      0           118.103636

[5 rows x 22 columns]
```

Assignment 7

Implement K-Means clustering on Iris.csv dataset. Determine the number

of clusters using the elbow method

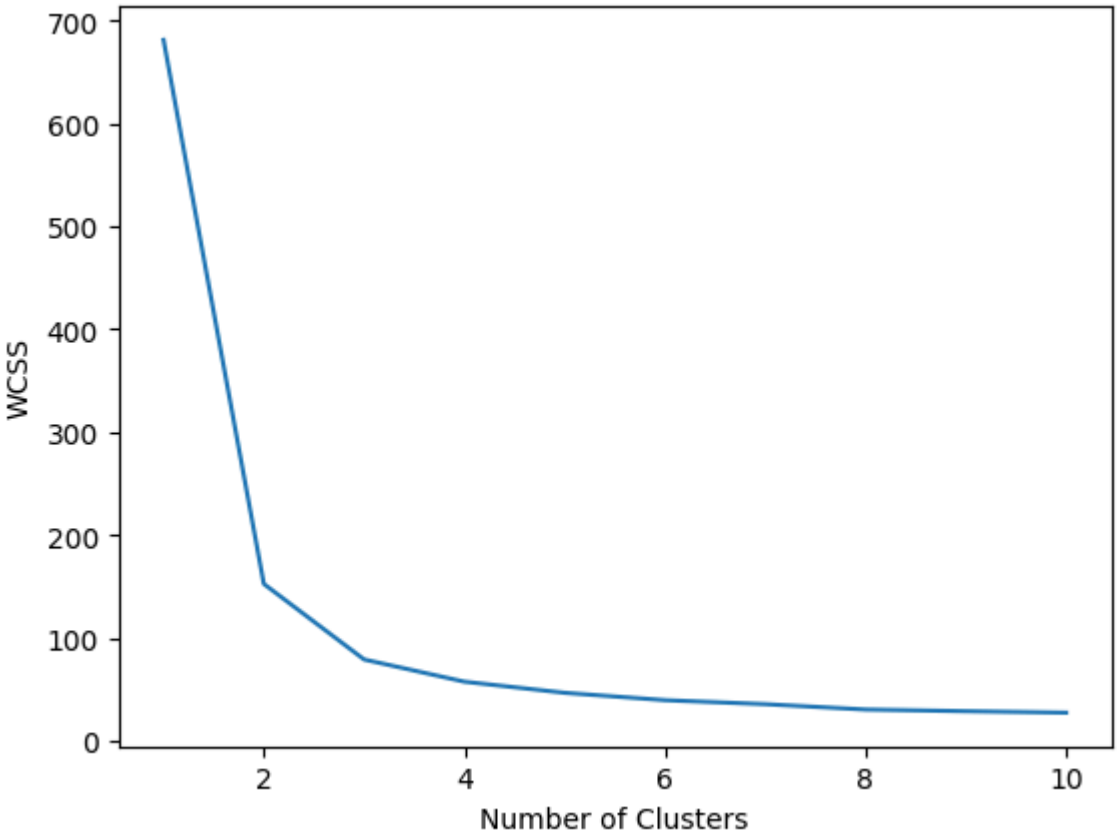
In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

df = pd.read_csv('Iris.csv')
X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]

wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k).fit(X)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss)
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()

kmeans = KMeans(n_clusters=3).fit(X)
print("Cluster Centers:\n", kmeans.cluster_centers_)
```




```
Cluster Centers:
 [[6.85384615 3.07692308 5.71538462 2.05384615]
 [5.006      3.428      1.462      0.246     ]
 [5.88360656 2.74098361 4.38852459 1.43442623]]
```

Assignment 8

Data Wrangling on Real Estate Mark

In [ ]:

```
import pandas as pd

# 1. Load dataset
df = pd.read_csv("RealEstate_Prices.csv")

# 2. Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# 3. Handle missing values
df = df.fillna(df.median(numeric_only=True))

# 4. Remove duplicates
df = df.drop_duplicates()

# 5. Remove price outliers
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)
IQR = Q3 - Q1

df = df[(df["price"] ⩾ Q1 - 1.5 * IQR) &
        (df["price"] ⩽ Q3 + 1.5 * IQR)]

# 6. Display average price by location
print("Average Price by Location:")
print(df.groupby("location")["price"].mean())

# 7. Convert text columns into numbers
df_encoded = pd.get_dummies(df, drop_first=True)

# 8. Save datasets
df.to_csv("cleaned_realestate.csv", index=False)
df_encoded.to_csv("encoded_realestate.csv", index=False)
```

```
Average Price by Location:
location
Mumbai    1.433791e+07
Nagpur    5.952219e+06
Nashik    6.842046e+06
Pune      8.653901e+06
Name: price, dtype: float64
```

Assignment 9

Ensemble Learning : A. Implement Random Forest Classifier model to pedict safety of car

In [ ]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("car.data")

# Input and output
X = data.drop(['safety', 'class'], axis=1)
y = data['safety']

# Convert text into numbers
X = pd.get_dummies(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train Random Forest
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Predict safety
y_pred = model.predict(X_test)

# Display results
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Predicted Safety:", y_pred[:10])
```

```
Accuracy: 0.02023121387283237
Predicted Safety: ['low' 'low' 'high' 'med' 'high' 'high' 'high' 'med'
'high' 'med']
```

Assignment 10

Analyzing Air Quality Index (AQI) Trends in a City. Create visualizations that effectively represent the AQI trends and patterns for different

pollutants in the city.

In [ ]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("City_Air_Quality.csv")

# Convert date and sort data
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

# Fill missing values
for col in ["AQI", "PM2.5", "PM10", "CO"]:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

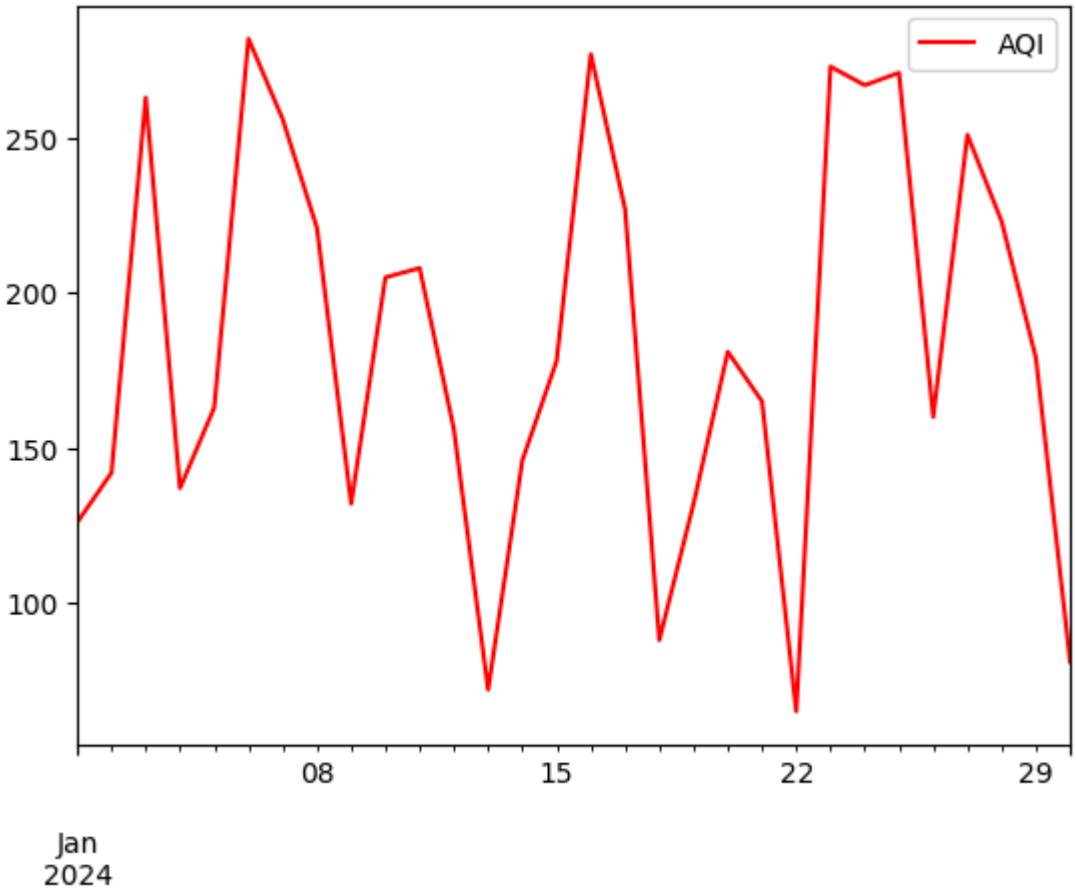
# AQI trend
df.plot(x="Date", y="AQI", color="red")
plt.show()

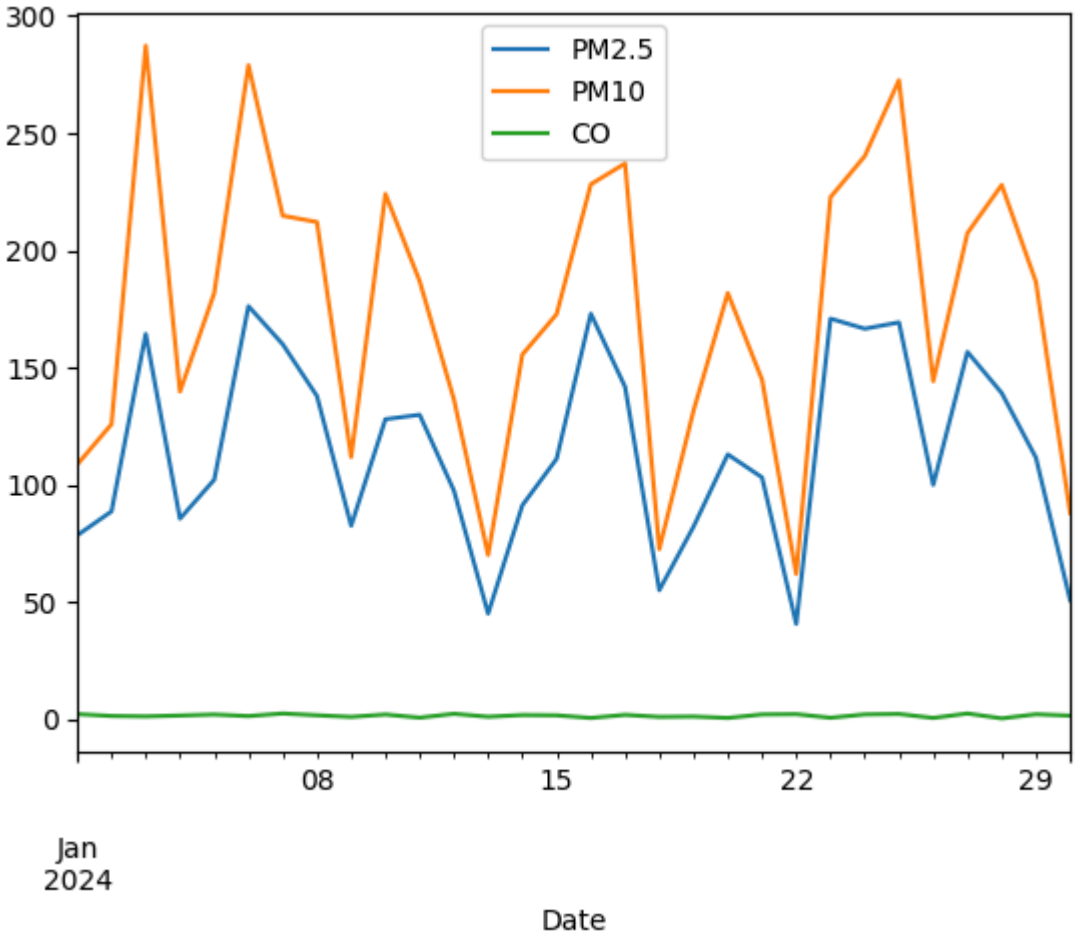
# Pollutant trends
df.plot(x="Date", y=["PM2.5", "PM10", "CO"])
plt.show()

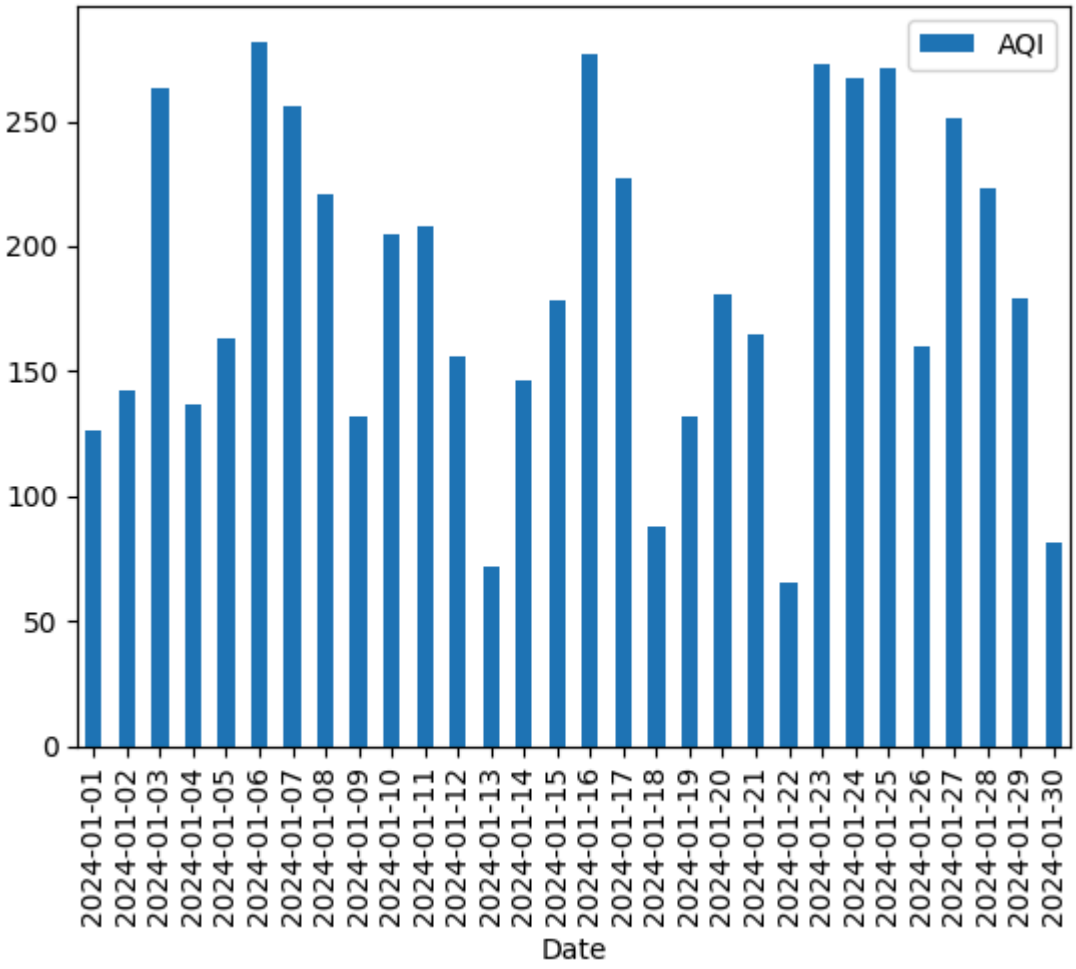
# AQI bar graph
df.assign(Date=df["Date"].dt.strftime("%Y-%m-%d")).plot(x="Date", y="AQI",
kind="bar")
plt.show()

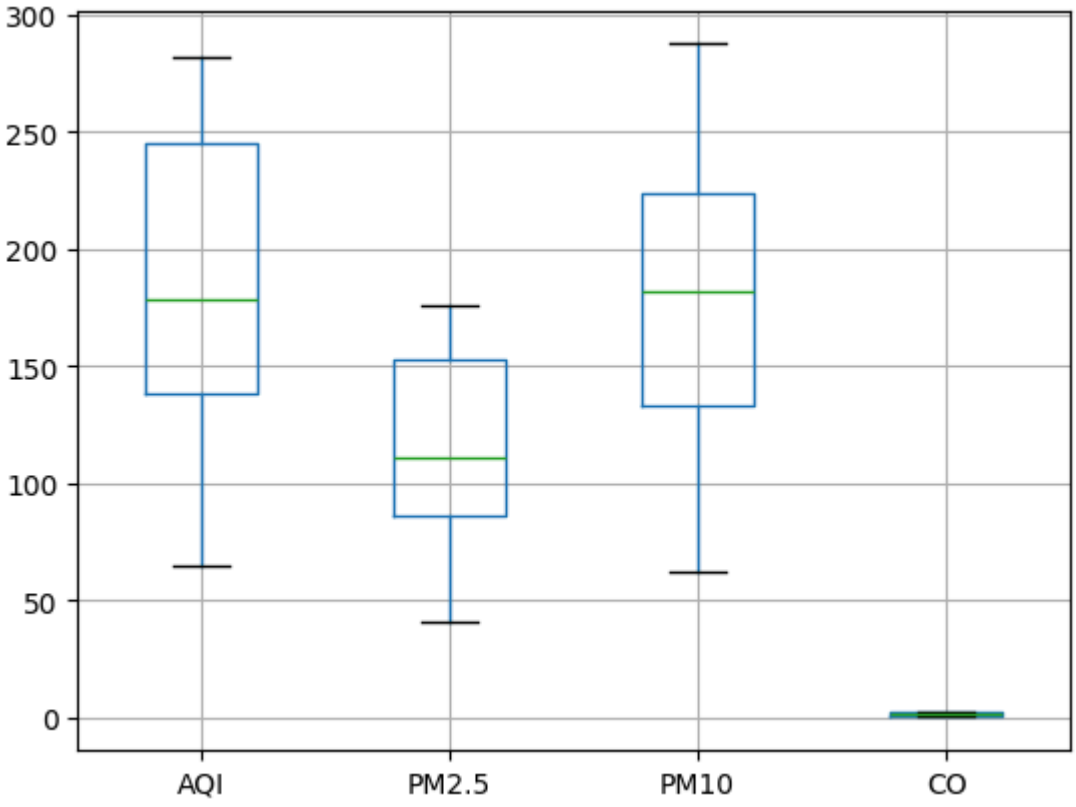
# Box plot of AQI and pollutants
df[["AQI", "PM2.5", "PM10", "CO"]].boxplot()
plt.show()

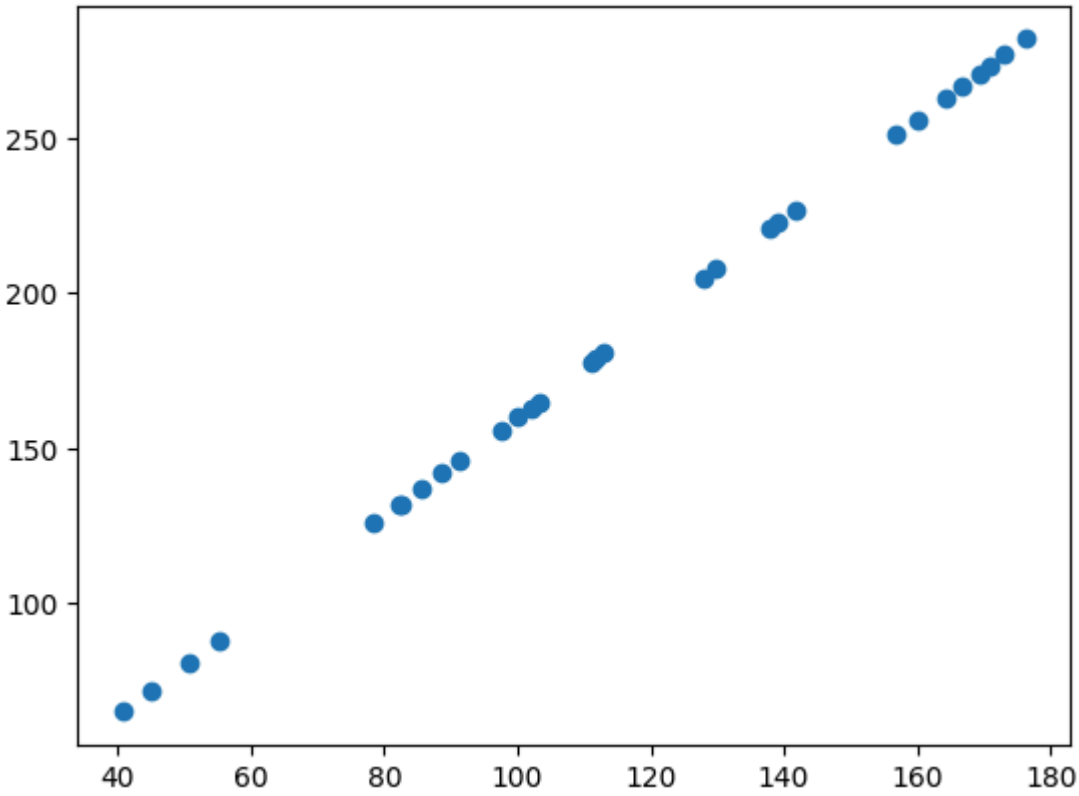
# AQI vs PM2.5 scatter plot
plt.scatter(df["PM2.5"], df["AQI"])
plt.show()
```


AQI
250
200
150
100
08
15
22
29
Jan
2024
Date


PM2.5
PM10
CO
300
250
200
150
100
50
0
08
15
22
29
Jan
2024
Date


AQI
250
200
150
100
50
0
2024-01-01
2024-01-02
2024-01-03
2024-01-04
2024-01-05
2024-01-06
2024-01-07
2024-01-08
2024-01-09
2024-01-10
2024-01-11
2024-01-12
2024-01-13
2024-01-14
2024-01-15
2024-01-16
2024-01-17
2024-01-18
2024-01-19
2024-01-20
2024-01-21
2024-01-22
2024-01-23
2024-01-24
2024-01-25
2024-01-26
2024-01-27
2024-01-28
2024-01-29
2024-01-30
Date


300
250
200
150
100
50
0
AQI
PM2.5
PM10
CO



Assignment 11

Implement Reinforcement Learning using an example of a maze environment that the agent needs to explore.

In [ ]:
```
import random

maze = [
    [0, 0, 0],
    [1, 0, 0],
    [0, 0, 2]
]

Q = [[0, 0] for _ in range(9)]

for _ in range(500):
    r, c = 0, 0

    for _ in range(20):
        if maze[r][c] == 2:
            break

        a = random.randint(0, 1)
        nr, nc = r, c

        if a == 0:
            nc += 1
        else:
            nr += 1
```

```
        if nr > 2 or nc > 2 or maze[nr][nc] == 1:
            reward, nr, nc = -5, r, c
        else:
            reward = 10 if maze[nr][nc] == 2 else -1

        s = r * 3 + c
        n = nr * 3 + nc

        Q[s][a] += 0.1 * (reward + 0.9 * max(Q[n]) - Q[s][a])

        r, c = nr, nc

print("Training completed!")
print("Q-table:")
for row in Q:
    print(row)
```

```
Training completed!
Q-table:
[4.579999995517376, -0.8780000109362157]
[6.199999997328063, 6.199964078454151]
[2.1999999971938977, 7.999999999885743]
[0, 0]
[7.999975120312802, 7.999746390615005]
[3.99999999999996, 9.999999999999993]
[0, 0]
[9.99998093162519, 3.99973471932409]
[0, 0]
```

Assignment 12

Analyzing Sales Performance by Region in a Retail Company. Perform data aggregation to analyze the sales performance by region and identify the

top-performing regions.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("Retail_Sales_Data.csv")

# Explore dataset
print(df.head())
print(df.info())
print(df.isnull().sum())

# Remove missing values
df = df.dropna(subset=["Region", "Product_Category", "Sales_Amount"])

# Calculate sales by region
s = df.groupby("Region")["Sales_Amount"].sum()
print(s)

# Plot sales by region
s.plot(kind="bar")
plt.show()

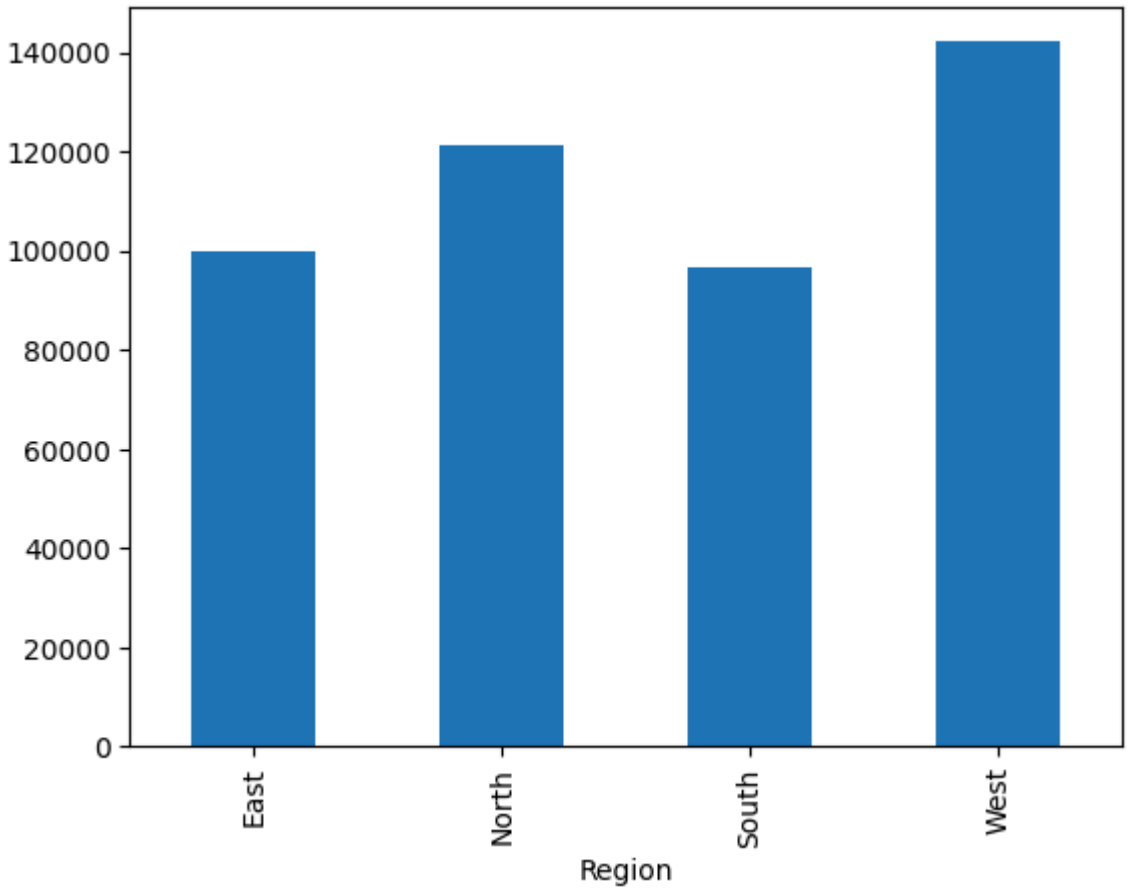
# Find top-performing region
print("Top Region:", s.idxmax())
print("Highest Sales:", s.max())

# Calculate sales by region and category
c = df.groupby(["Region", "Product_Category"])
["Sales_Amount"].sum().unstack()
print(c)

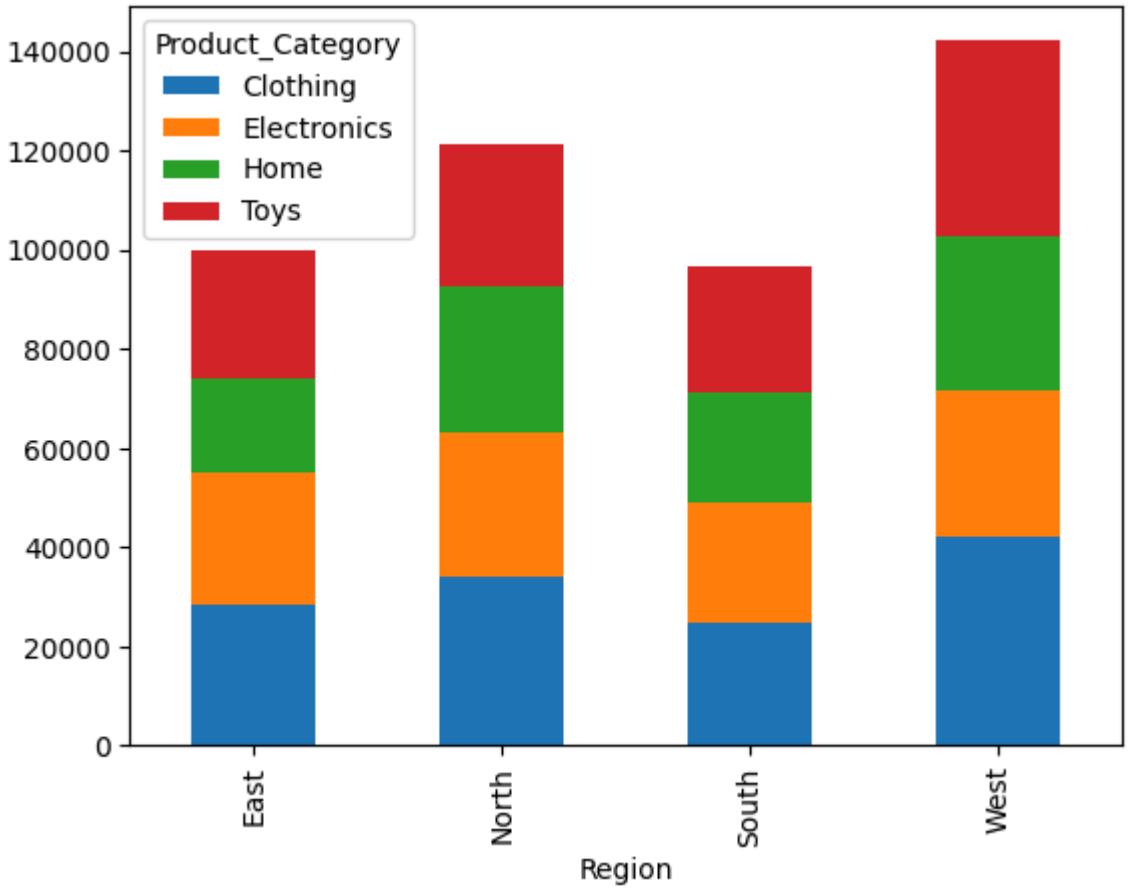
# Plot stacked bar chart
c.plot(kind="bar", stacked=True)
plt.show()
```

```
  Region Product_Category  Sales_Amount
0   West      Electronics       1656.59
1  South             Home       1630.75
2  South         Clothing       1981.25
3   West             Toys       2131.58
4   East             Toys       1683.70
<class 'pandas.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 3 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Region            300 non-null    str
 1   Product_Category  300 non-null    str
 2   Sales_Amount      300 non-null    float64
dtypes: float64(1), str(2)
```

```
memory usage: 7.2 KB
None
Region              0
Product_Category    0
Sales_Amount        0
dtype: int64
Region
East     100097.40
North    121478.63
South     96737.21
West     142161.76
Name: Sales_Amount, dtype: float64
```




```
Top Region: West
Highest Sales: 142161.76
Product_Category  Clothing  Electronics      Home      Toys
Region
East              28632.89     26519.17  18807.78  26137.56
North             33972.12     29378.34  29448.13  28680.04
South             24864.45     24179.90  22094.10  25598.76
West              42347.14     29309.10  31185.75  39319.77
```



Assignment 13

Analysis and Visualization of Stock Market Data.

The goal is to perform time series data analysis on the stock price data to identify trends, patterns, and potential predictors, as well as build models to forecast future stock prices.

In [ ]:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

# 1. Load dataset
df = pd.read_csv("Stock_Prices.csv")
df["Date"] = pd.to_datetime(df["Date"])
df = df.dropna(subset=["Date", "Close"]).sort_values("Date")
df.set_index("Date", inplace=True)

# 2. Plot stock prices
df["Close"].plot()
plt.show()

# 3. Moving Average
df["Close"].rolling(7).mean().plot()
plt.show()

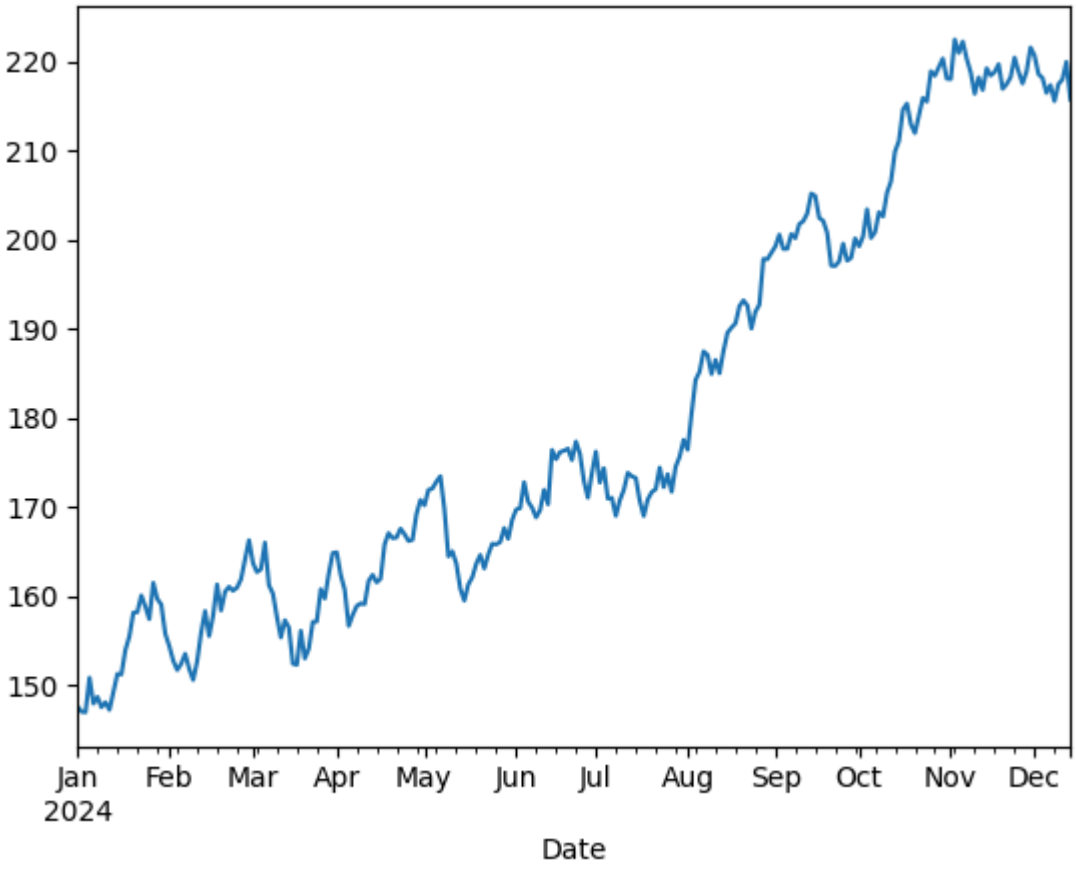
# 4. Monthly average
df["Close"].resample("ME").mean().plot()
plt.show()

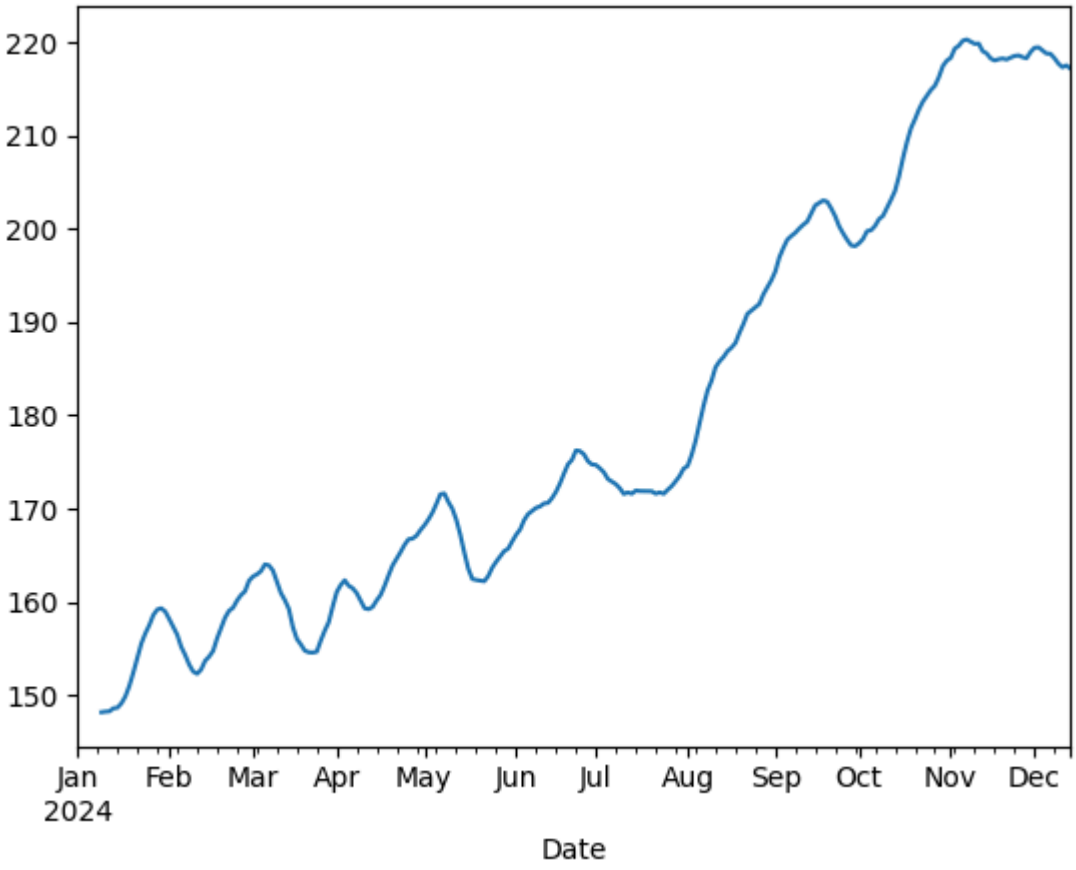
# 5. Correlation
print(df.corr(numeric_only=True))

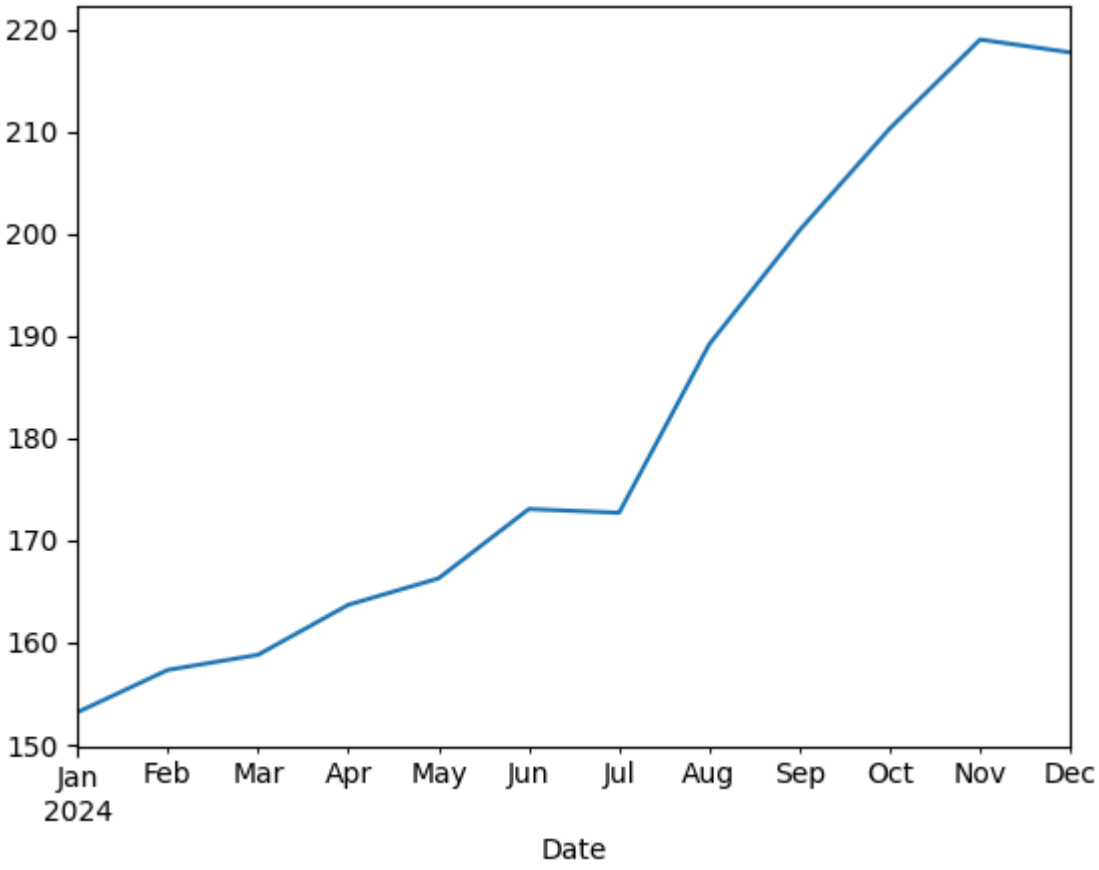
# 6. ARIMA Forecast
model = ARIMA(df["Close"], order=(1, 1, 1)).fit()
forecast = model.forecast(steps=5)

print("Next 5 Forecasted Prices:")
print(forecast)

forecast.plot()     #stock price forecast
plt.show()
```

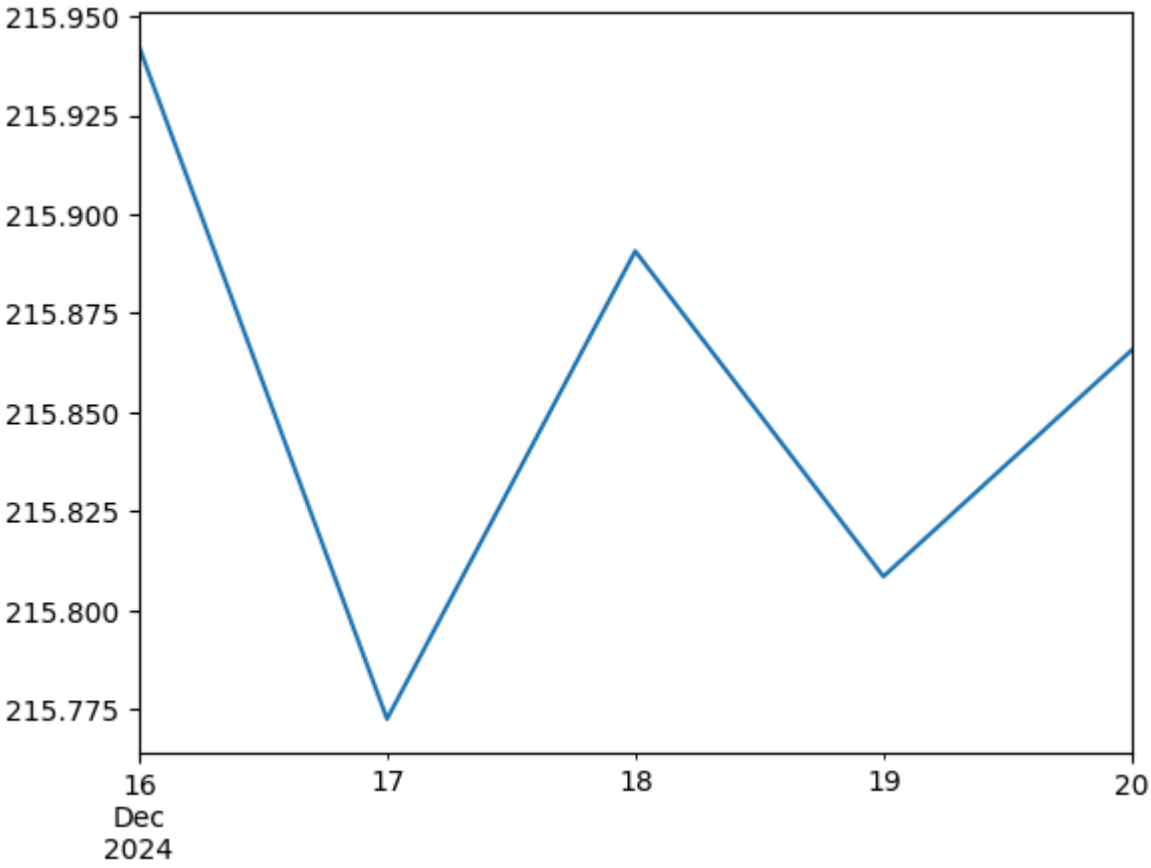

220
210
200
190
180
170
160
150
Jan
2024
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Date


220
210
200
190
180
170
160
150
Jan
2024
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Date



	Open	High	Low	Close	Volume
Open	1.000000	0.999519	0.999529	0.999770	-0.009740
High	0.999519	1.000000	0.999566	0.999762	-0.010377
Low	0.999529	0.999566	1.000000	0.999778	-0.011479
Close	0.999770	0.999762	0.999778	1.000000	-0.011325
Volume	-0.009740	-0.010377	-0.011479	-0.011325	1.000000

```
Next 5 Forecasted Prices:
2024-12-16    215.942484
2024-12-17    215.772534
2024-12-18    215.890720
2024-12-19    215.808532
2024-12-20    215.865687
Freq: B, Name: predicted_mean, dtype: float64
```


215.950
215.925
215.900
215.875
215.850
215.825
215.800
215.775
16
Dec
2024
17
18
19
20